

B28: SQL SuperHero Program : Session 18

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

13 March 2025 at 22:56

```
create table runners (id number, name varchar2(20));
```

```
insert into runners values (1,'John Doe');  
insert into runners values (2,'Jane Doe');  
insert into runners values (3,'Alice jones');  
insert into runners values (4,'abcd');  
insert into runners values (5,'xyz');
```

```
create table races (id number, event varchar2(20), winner_id number);
```

```
insert into races values (1,'100 meter', 2);  
insert into races values (2,'500 meter', 3);  
insert into races values (3,'cross -country', 2);  
insert into races values (4,'marathon', null);
```

1,4,5
never
won
any
race

ID	NAME
1	John Doe
2	Jane Doe
3	Alice jones
4	abcd
5	xyz

Download CSV
5 rows selected.

ID	EVENT	WINNER_ID
1	100 meter	2
2	500 meter	3
3	cross -country	2
4	marathon	-

2,3
are
winner
who
won the
race

[Understand this question here :](#)

Find out all the runners who won a race

If we see both the tables, we can see that races tables contains a column called , winner_id which is actually contains the id of table runners. whoever win the race that runner information saved a swinner_id .
so, if we write a query like that:

```
select * from runners where id in (select winner_id from races);
```

```

1 select * from runners where id in (select winner_id from races);
2 |

```

ID	NAME
2	Jane Doe
3	Alice jones

Download CSV

2 rows selected.

✓ correct ✓

It means we just check all such runners, whose id present in races tables as winner_id. Lets change the same query using EXISTS.

```
select * from runners RU where exists (select 1 from races RA Where RA.Winner_id = RU.Id);
```

This is how we write, EXISTS. Exists will return boolean , just check data there or not. in case of IN operator **where id in ()** , we need 2 things to compare, because IN is just an extension of = , but for EXISTS we are not comparing.

```
SELECT * FROM RUNNERS RU WHERE EXISTS (SELECT 1 FROM RACES RA WHERE RU.ID = RA.WINNER_ID)
```

Script Output x Query Result 2 x Query Result 3 x Query Result 4 x

SQL | All Rows Fetched: 2 in 0.012 seconds

ID	NAME
1	2 Jane Doe
2	3 Alice jones

✓ correct

Lets understand the trick that is NOT IN VS NOT EXISTS

Find out all the runners who never won a race.

It means that runners id , not present in race table as winner_id.

```
select * from runners RU where not exists (select 1 from races RA Where RA.Winner_id = RU.Id);
```

unable to get correct ans

```
SELECT * FROM RUNNERS WHERE ID NOT IN (SELECT WINNER_ID FROM RACES);
```

Script Output x | Query Result 2 x | Query Result 3 x | Query Result 4 x

SQL | All Rows Fetched: 0 in 0.003 seconds

ID	NAME
----	------

✓ no data → its wrong

NOT IN can not get any data, because the subquery returns null and NOT in will not work with NULL. lets change with NOT EXISTS

Helps when null data

```
SELECT * FROM RUNNERS RU WHERE Not EXISTS (SELECT 1 FROM RACES RA WHERE RU.ID = RA.WINNER_ID)
```

Script Output x | Query Result 2 x | Query Result 3 x | Query Result 4 x

SQL | All Rows Fetched: 3 in 0.006 seconds

ID	NAME
1	1 John Doe
2	4 abcd
3	5 xyz

✓ correct

the NOT EXISTS and NOT IN behave differently when there are null values involved.

The advantage of using the SQL EXISTS and NOT EXISTS operators is that the inner subquery execution can be stopped as long as a matching record is found.

If the subquery requires to scan a large volume of records, stopping the subquery execution as soon as a single record is matched can greatly speed up the overall query response time.

=====

The EXISTS Operator

The EXISTS operator is used to check if existence of any record in a subquery.

The result of this operator is TRUE or FALSE.

The NOT EXISTS Operator

The NOT EXISTS operator is just like EXISTS but it is REVERSE of EXISTS.

The result of this operator is TRUE or FALSE.

An EXISTS condition tests for existence of rows in a subquery. If at least one row returns, it will evaluate as TRUE. NOT EXISTS evaluates as TRUE if 0 rows are returned and can be used to validate the absence of a condition.

```
select * from scott.emp e1 where not exists ( select 1 from scott.emp e2 where e2.mgr = e1.empno );
```

See the above query, it will give you all employees (in the EMP table) that are NOT managers. It checks for their employee number as a manager (mgr column) and returns them if they are not found.

--> Whenever see NULL values, never use NOT IN, always use NOT EXISTS

The EXISTS clause is much faster than IN when the subquery results is very large. Conversely, the IN clause is faster than EXISTS when the subquery results is very small. Also, the IN clause can't compare anything with NULL values, but the EXISTS clause can compare everything with NULLs.

=====

```
create table departments as select * from hr.departments
```

Table created.

```
create table locations as select * from hr.locations
```

Table created.

```
select * from locations where LOCATION_ID not in (select location_id from departments) ';
```

→ All location which are not in dept table

```
select * from locations where location_id not in (select location_id from departments)
```

we can see my data

LOCATION_ID	STREET_ADDRESS	POSTAL_CODE	CITY	STATE_PROVINCE	COUNTRY_ID
2200	12-98 Victoria Street	2901	Sydney	New South Wales	AU
2300	198 Clementi North	540198	Singapore	-	SG
1000	1297 Via Cola di Rie	00989	Roma	-	IT
1600	2007 Zagora St	50090	South Brunswick	New Jersey	US
3100	Pieter Breughelstraat 837	3029SK	Utrecht	Utrecht	NL
2600	9702 Chester Road	09629850293	Stretford	Manchester	UK
2900	20 Rue des Corps-Saints	1730	Geneva	Geneve	CH

→ 1 Record set to NULL

```
update departments set location_id = null where LOCATION_ID = 1500
```

1 row(s) updated.

```
select * from locations where LOCATION_ID not in (select location_id from departments)
```

no data found

→ enable to compare

Now can see data

```
select * from locations l where not exists (select 1 from departments d where d.location_id = l.location_id)
```

LOCATION_ID	STREET_ADDRESS	POSTAL_CODE	CITY	STATE_PROVINCE	COUNTRY_ID
2200	12-98 Victoria Street	2901	Sydney	New South Wales	AU
2300	198 Clementi North	540198	Singapore	-	SG
1000	1297 Via Cola di Rie	00989	Roma	-	IT
1600	2007 Zagora St	50090	South Brunswick	New Jersey	US
3100	Pieter Breughelstraat 837	3029SK	Utrecht	Utrecht	NL
2600	9702 Chester Road	09629850293	Stretford	Manchester	UK
2900	20 Rue des Corps-Saints	1730	Geneva	Geneve	CH
1500	2011 Interiors Blvd	99236	South San Francisco	California	US
3200	Mariano Escobedo 9991	11932	Mexico City	Distrito Federal,	MX
2800	Rua Frei Caneca 1360	01307-002	Sao Paulo	Sao Paulo	BR
1100	83901 Calle della Testa	10034	Venice	-	IT

If you see the above then you can realize that when my inner query have NULL then NOT IN does not give correct data.

=====

Below Example from google:

The student table contains the following two records:

id	first_name	last_name	admission_score
1	Alice	Smith	8.95
2	Bob	Johnson	8.75

And, the student_grade table stores the grades the students received:

id	class_name	grade	student_id
1	Math	10	1
2	Math	9.5	1
3	Math	9.75	1
4	Science	9.5	1
5	Science	9	1
6	Science	9.25	1
7	Math	8.5	2
8	Math	9.5	2
9	Math	9	2
10	Science	10	2
11	Science	9.4	2

filter the student records that have a 10 grade in Math

```
SELECT
    id, first_name, last_name
FROM
    student
WHERE EXISTS (
    SELECT 1
    FROM
        student_grade
    WHERE
        student_grade.student_id = student.id AND
        student_grade.grade = 10 AND
        student_grade.class_name = 'Math'
);
```

all student records that have no associated student_grade with a value lower than 9

```
SELECT
    id, first_name, last_name
FROM
    student
WHERE NOT EXISTS (
    SELECT 1
    FROM
        student_grade
    WHERE
        student_grade.student_id = student.id AND
        student_grade.grade < 9
);
```

The advantage of using the SQL EXISTS and NOT EXISTS operators is that the inner subquery execution can be stopped as long as a matching record is found.

If the subquery requires to scan a large volume of records, stopping the subquery execution as soon as a single record is matched can greatly speed up the overall query response time.